

Weather-Aware Smart Planner

A Python-Based Task Scheduling System with Weather Forecast
Integration and Machine Learning

VITyarthi — Build Your Own Project
Project Report

Submitted by: Rishabh Khandelwal

Program: B.Tech, Computer Science Engineering (AI/ML Specialization)

GitHub Repository: github.com/rishabhkhandelwal322-Igtn/Weather-Aware-Smart-Planner

1. Introduction

The Weather-Aware Smart Planner is a Python application designed to help individuals manage their daily tasks in a way that accounts for real-world weather conditions. Traditional to-do list and calendar applications treat time as the only planning variable, which frequently leads to disrupted schedules when outdoor-dependent tasks are planned on days with unfavorable weather. This project addresses that gap by combining live weather forecast data, a rule-based rescheduling engine, a machine learning prediction layer, and a productivity analytics dashboard into a single, cohesive system.

The system is built entirely in Python, using SQLite for persistent storage, the OpenWeatherMap API for live weather data, scikit-learn for a probabilistic prediction model, and Streamlit for an interactive front-end dashboard. The project demonstrates the practical application of core programming concepts — API integration, relational data modeling, rule-based automation, machine learning, and user interface design — within a single, functioning application.

2. Problem Statement

People frequently plan outdoor tasks — errands, workouts, commutes, events — without factoring in short-term weather changes, leading to disrupted schedules, wasted time, and missed opportunities to take advantage of favorable weather windows. Existing calendar and to-do applications treat time as the only planning variable and ignore environmental conditions entirely. There is a clear need for a planning tool that actively incorporates weather forecasts and trends into task scheduling decisions, rather than leaving that correlation entirely to the user's manual judgment.

This project is scoped as a single-user, local desktop/web application (delivered via Streamlit) and does not include multi-user accounts, native mobile apps, or integration with third-party calendar services in this iteration. The target users are students and individuals who plan outdoor-dependent tasks and want a lightweight, automated assistant that reduces the manual effort of checking forecasts and rescheduling tasks by hand.

3. Functional Requirements

Task Management (CRUD): Users can create, view, filter, update, complete, and delete tasks. Each task is tagged as 'outdoor' or 'indoor', with a priority level, an optional deadline, and a scheduled date.

Live Weather Forecast Retrieval: The system fetches current conditions and a 5-day/3-hour forecast from the OpenWeatherMap API, aggregates the 3-hour blocks into daily summaries, and caches them locally to minimize redundant API calls.

Manual Forecast Entry: Users can manually insert or overwrite forecast data for a given date and location without depending on the live API — useful for testing, offline use, or covering gaps in the 5-day API window.

Rule-Based Rescheduling Engine: Outdoor tasks scheduled on a day with a rain probability above a defined threshold are automatically identified and moved to the next available day within the forecast window that meets the favorable-weather criteria.

ML-Based Outdoor Window Prediction: A trained scikit-learn classification model estimates the probability (0–100%) that a given day is favorable for outdoor activity, based on temperature, rain probability, wind speed, and seasonal month — providing a smoother, more nuanced signal than the rule-based threshold alone.

Notifications: The system sends desktop notifications (via plyer) and, when configured, email notifications (via SMTP) whenever a task is automatically rescheduled or cannot be rescheduled within the forecast window.

Productivity Analytics: The system computes and visualizes task completion rates overall, by task type (outdoor vs indoor), and correlated against weather conditions (rainy vs clear days), producing chart images for reporting.

Interactive Dashboard: A Streamlit-based web dashboard ties all the above together into a single interface with dedicated tabs for tasks, forecast, analytics, and activity history.

4. Non-Functional Requirements

Requirement	How It Is Addressed
Performance	Forecast data is cached in SQLite (one row per location/date) so repeated planning cycles within the same day do not re-hit the external API.
Reliability	All task and forecast data is persisted in a local SQLite database (planner.db), surviving application restarts. Every state-changing operation is wrapped in a transactional connection context manager with rollback on failure.
Security	API keys and SMTP credentials are stored in a local .env file, loaded via python-dotenv, and excluded from version control via .gitignore — never hardcoded in source.
Usability	The Streamlit dashboard provides a clear, tabbed interface requiring no configuration to view tasks; sensible defaults (medium priority, current date) reduce data-entry friction.
Scalability	The modular design (separate storage, fetcher, planner, predictor, notifier, and analytics modules) allows additional locations, users, or forecast sources to be added without restructuring the core system.
Maintainability	Each module has a single, well-defined responsibility with docstrings and type-consistent function signatures; the test suite (tests/) validates core planner and task-manager behavior independently.
Error Handling	Custom exception classes (WeatherFetchError, ValidationError, NotificationError) are raised for expected failure modes and caught at the UI/CLI boundary, so a missing API key or invalid input never crashes the application outright.

5. System Architecture

The system follows a layered architecture. A presentation layer (the Streamlit dashboard, plus a CLI entry point via `task_manager.py`) sits above an application logic layer containing the task manager, planner engine, ML predictor, and analytics modules. These depend on two support services — the weather fetcher and the notifier — which in turn depend on a single data layer (`storage.py`) backed by a local SQLite database. External dependencies are limited to the OpenWeatherMap API and the operating system's notification/SMTP facilities.

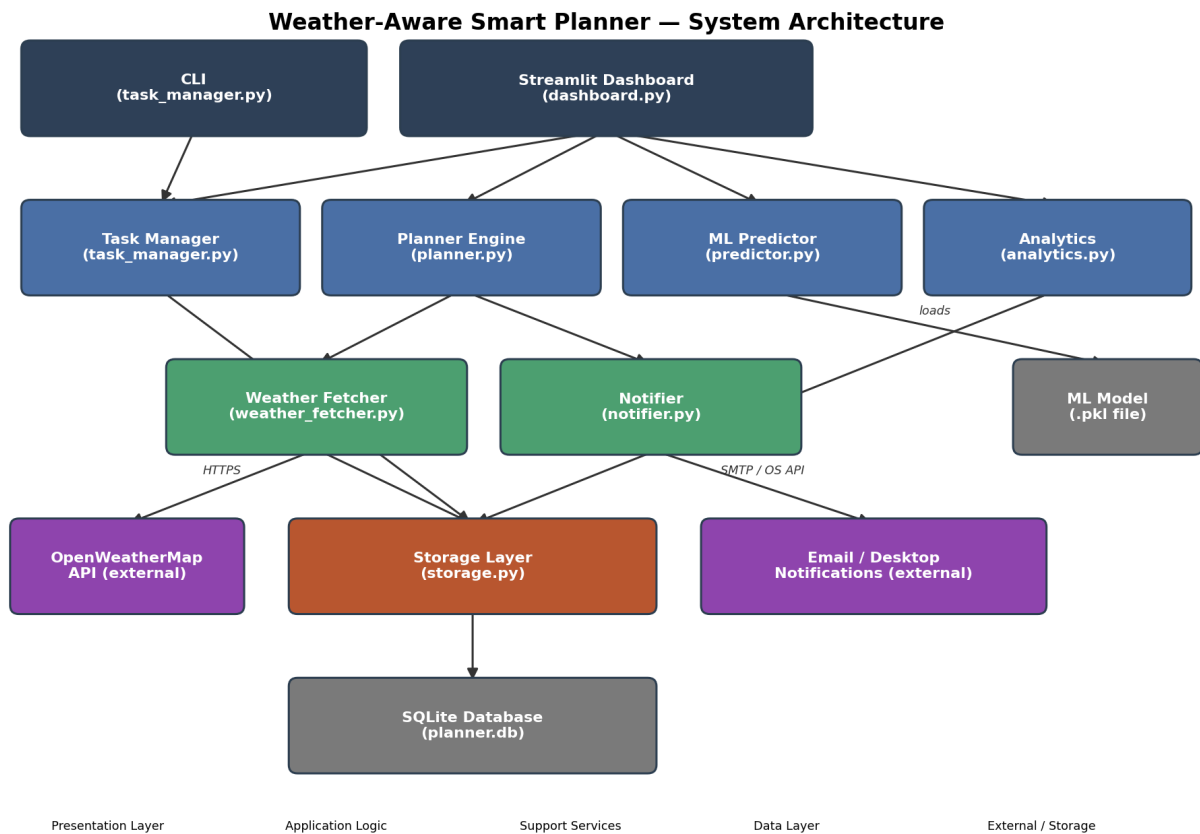


Figure 5.1 — System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

The single actor (User) interacts with eleven use cases spanning task management, forecast handling, planning automation, notifications, and analytics.

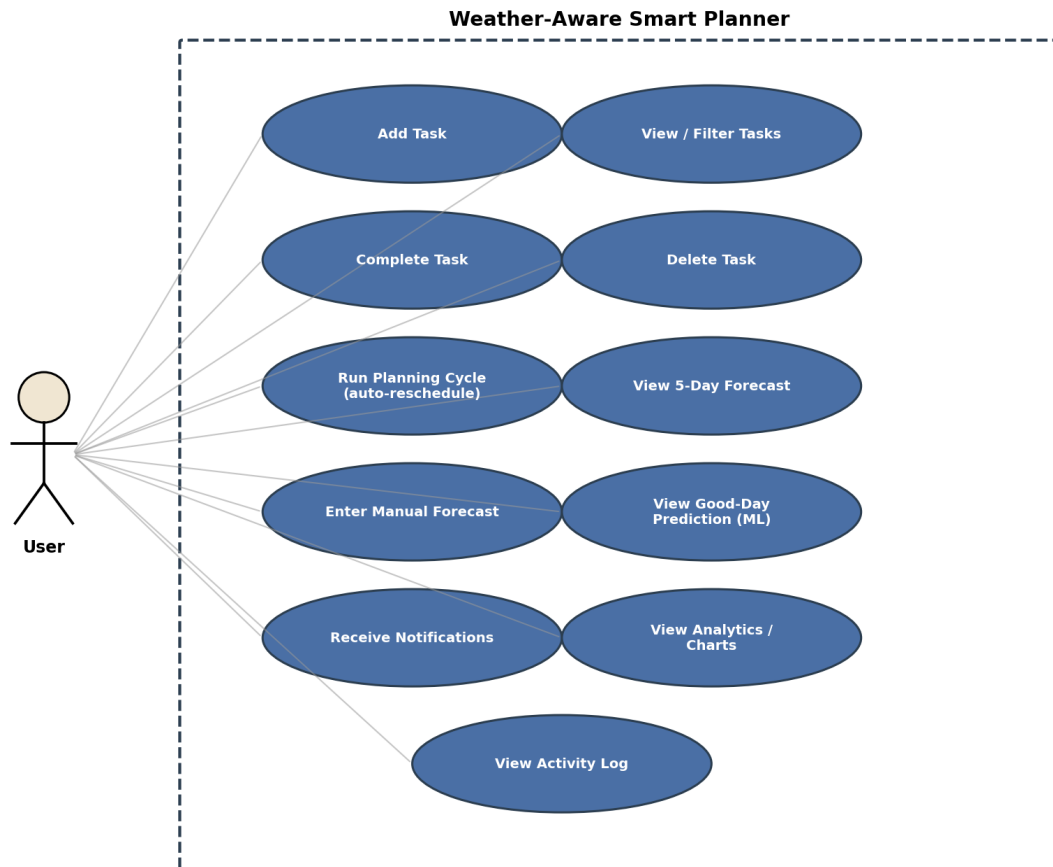


Figure 6.1 — Use Case Diagram

6.2 Workflow / Process Flow Diagram

This diagram illustrates the core rescheduling logic executed each time a planning cycle runs: each outdoor task's scheduled date is checked against the cached forecast, and tasks landing on a high-rain-probability day are automatically moved to the next available favorable day, with a notification sent on success and an unresolved flag raised if no suitable day exists in the window.

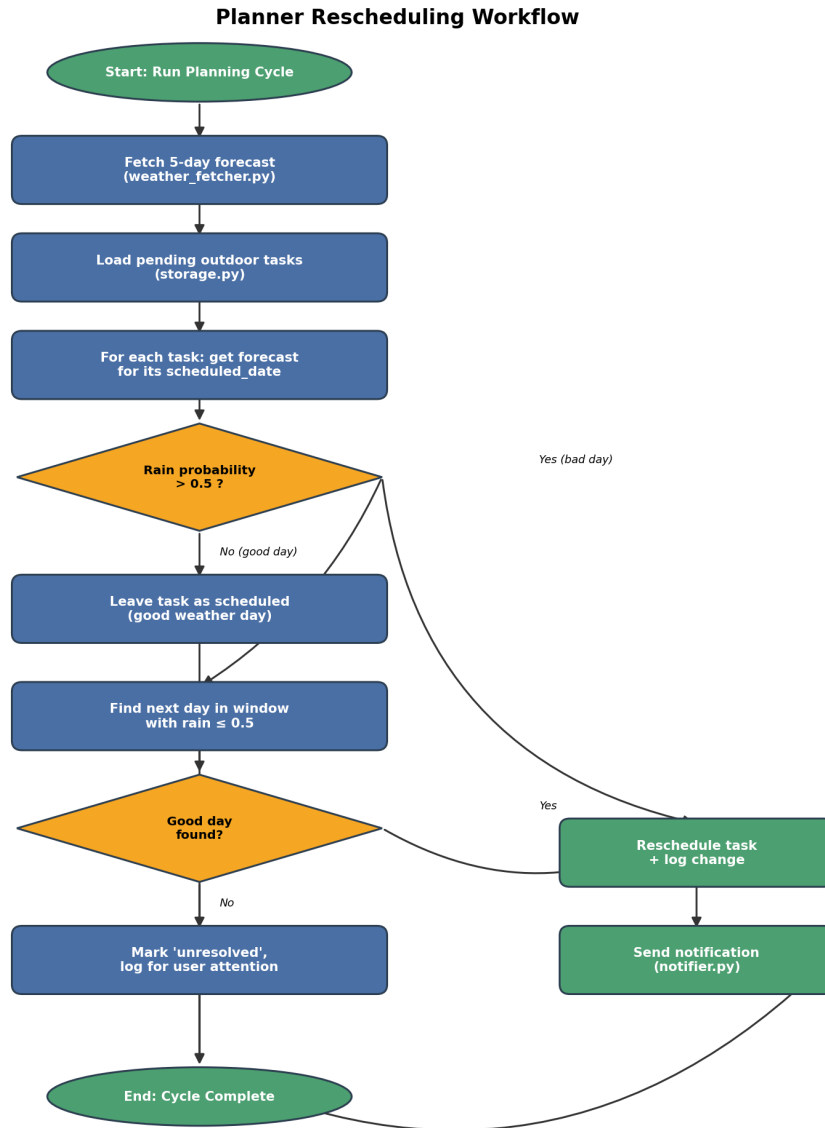


Figure 6.2 — Planner Rescheduling Workflow

6.3 Sequence Diagram

The sequence diagram below traces a single 'Run Planning Cycle' interaction from the user clicking the button in the dashboard through to the forecast fetch (with cache check), task evaluation, rescheduling, and notification dispatch.

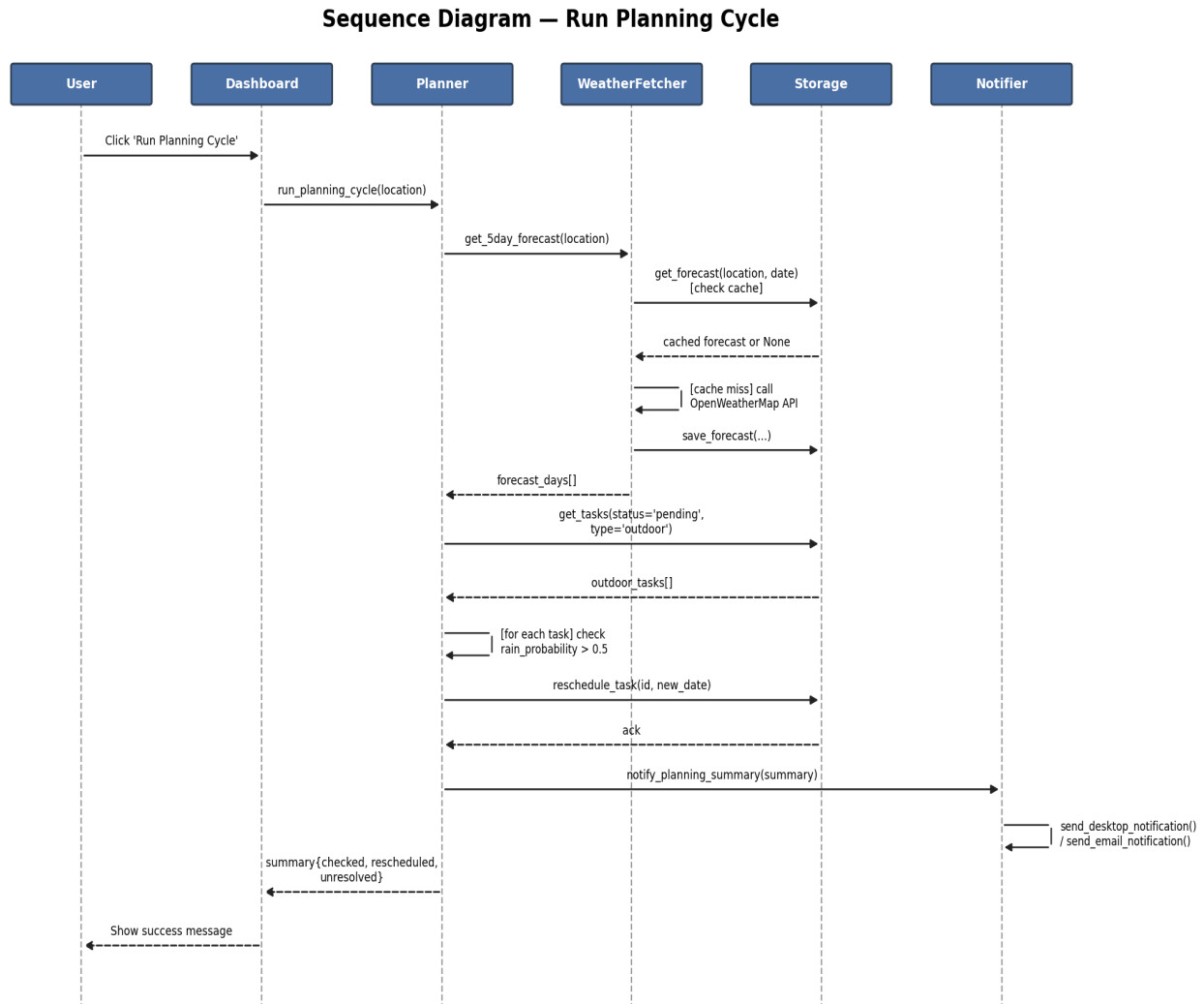


Figure 6.3 — Sequence Diagram: Run Planning Cycle

6.4 Class / Component Diagram

Each Python module is represented as a UML-style class box showing its key attributes and public functions, with dependency arrows showing which modules the dashboard orchestrates and how data flows down to the shared storage layer.

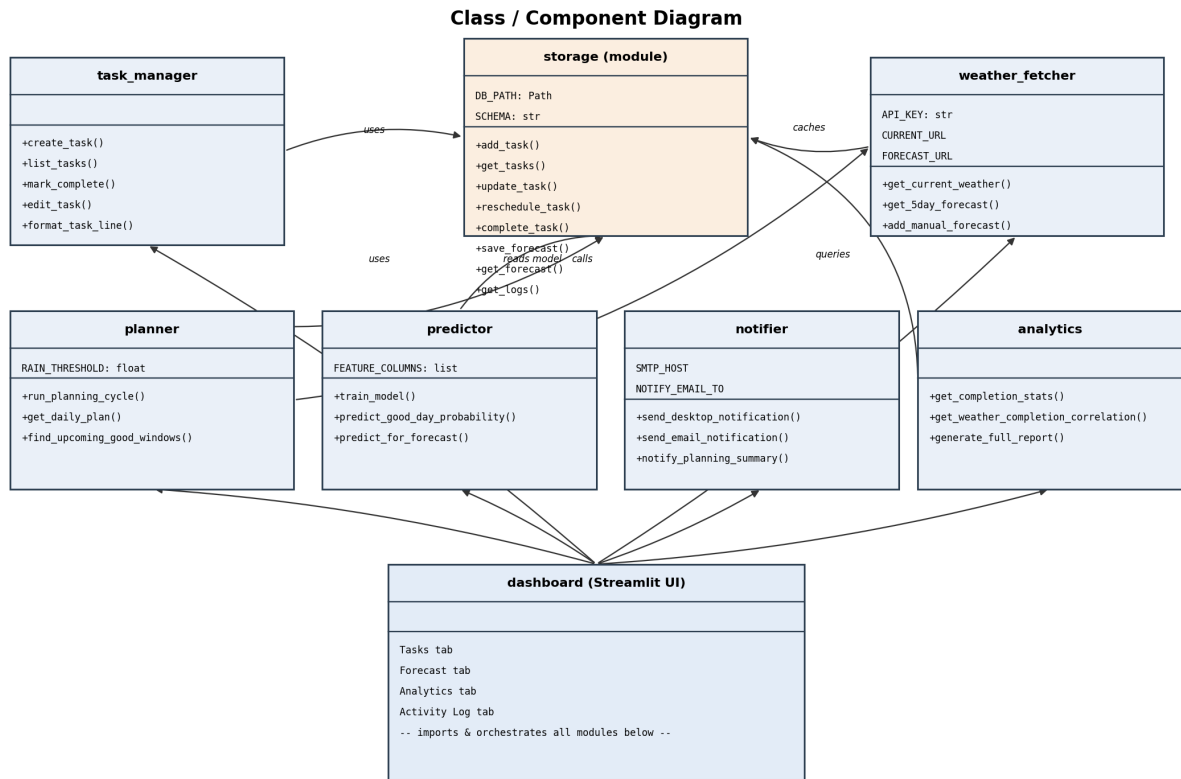


Figure 6.4 — Class / Component Diagram

6.5 ER Diagram / Database Schema

The SQLite database consists of three tables: tasks, forecasts, and logs. The logs table has a formal foreign-key relationship to tasks (one task generates zero or more log entries). The forecasts table has no formal foreign key to tasks; instead, the planner joins them logically at the application level by matching a task's scheduled_date against a forecast's (location, forecast_date) pair.

ER Diagram — Weather-Aware Smart Planner

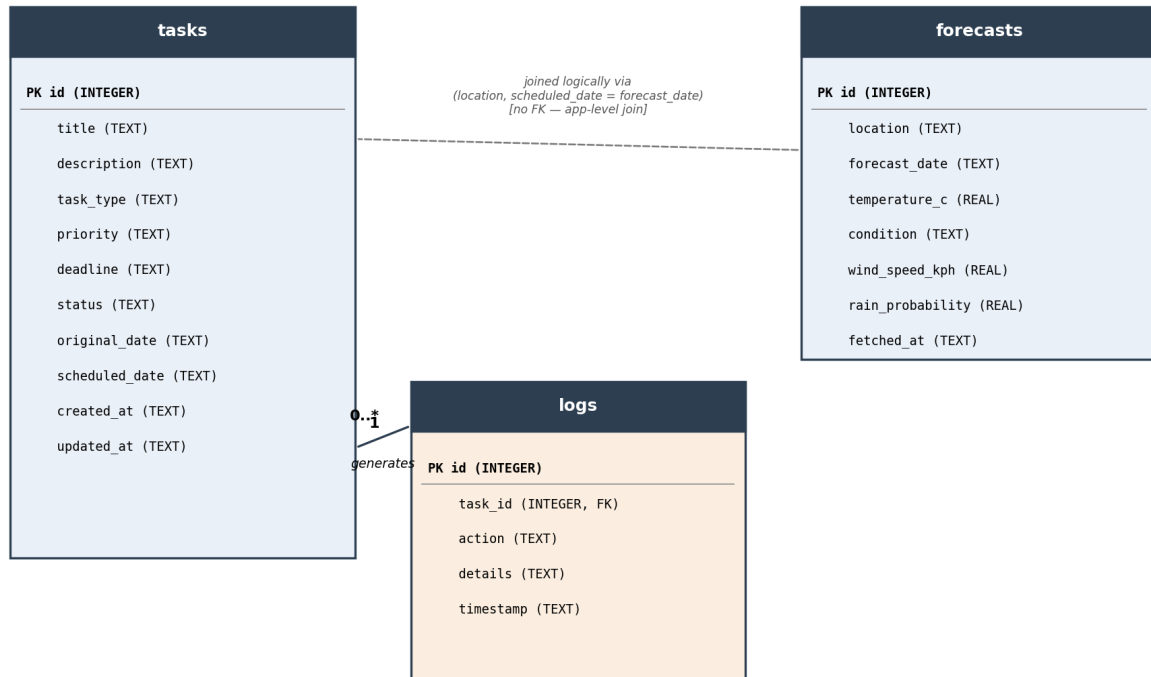


Figure 6.5 — Entity-Relationship Diagram

7. Design Decisions & Rationale

SQLite over a JSON file or a full DBMS: SQLite provides real relational structure (enabling the tasks—logs foreign key and SQL-based filtering/aggregation for analytics) without requiring a separate database server, keeping the project fully self-contained and easy to run for evaluation.

Rule-based rescheduling and ML prediction, in combination: The planner's rule-based threshold (rain probability > 0.5) is deterministic, auditable, and simple to explain and test. The ML predictor was added on top as a complementary, probabilistic signal (0-100% confidence) that learns smoother decision boundaries across temperature, wind, and seasonality — offering a richer signal in the dashboard without replacing the transparent rule that actually drives rescheduling.

Synthetic training data for the ML model: No multi-year historical weather log was available for this project's target location. A synthetic dataset was generated with realistic seasonal temperature curves, monsoon-aware rain probability distributions, and deliberate label noise (7%) so the model learns generalizable patterns rather than memorizing a hand-written rule.

Caching forecast data rather than fetching on every call: The OpenWeatherMap free tier has request-rate constraints, and repeated calls within the same day would be wasteful. Caching by (location, date) in the forecasts table means a full planning cycle costs at most one API call per uncached day.

Streamlit for the dashboard instead of Tkinter or Flask: Streamlit allows a full interactive multi-tab web UI to be built directly in Python with minimal boilerplate, making it well suited to a project of this scope compared to hand-writing HTML/CSS/JS for Flask or building a native Tkinter GUI.

Graceful degradation for optional features: Desktop and email notifications, and the ML predictor, are all designed to fail silently (returning a status rather than raising) when not configured or not yet trained, so the core task-and-planner functionality never depends on them being present.

8. Implementation Details

The project is implemented as eleven Python modules under a `src/` package, totaling roughly 2,100 lines of application code excluding tests and generated assets. Source code, tests, and generated data/artifacts are kept in separate top-level folders (`src/`, `tests/`, `data/`, `models/`, `reports/`) following standard Python project conventions.

Module	Responsibility	Key Libraries
<code>src/storage.py</code>	SQLite schema, connection management, CRUD for tasks/forecasts/logs	sqlite3
<code>src/weather_fetcher.py</code>	Live API fetch + aggregation, caching, manual forecast entry	requests, python-dotenv
<code>src/planner.py</code>	Rule-based rescheduling engine, good-window lookup	storage, weather_fetcher
<code>src/predictor.py</code>	Synthetic data generation, model training, probability prediction	scikit-learn, pandas, joblib
<code>src/task_manager.py</code>	Input validation, task CRUD wrapper, CLI interface	argparse
<code>src/notifier.py</code>	Desktop and email notifications with graceful fallback	plyer, smtplib

src/analytics.py	Completion-rate stats, weather correlation, chart generation	matplotlib
src/dashboard.py	Streamlit UI tying every module together (4 tabs)	streamlit
src/run_interactive.py	Menu-driven terminal interface for non-technical users; renders colored bar/pie charts in-terminal and can pop up chart images in a window	termcharts, rich, matplotlib
src/seed_demo_data.py	One-off script to populate realistic sample tasks/forecasts for demos	storage

Example: the core rescheduling logic (src/planner.py)

```
def run_planning_cycle(location):
    forecast_days = weather_fetcher.get_5day_forecast(location)
    forecast_by_date = {f["forecast_date"]: f for f in forecast_days}
    outdoor_tasks = storage.get_tasks(status="pending", task_type="outdoor")

    for task in outdoor_tasks:
        forecast = forecast_by_date.get(task["scheduled_date"])
        if not _is_bad_weather_day(forecast):
            continue # already on a good day
        new_date = _find_next_good_day(forecast_by_date, after_date=task["scheduled_date"])
        if new_date:
            storage.reschedule_task(task["id"], new_date, reason="...")
```

9. Screenshots / Results

The following charts are generated by analytics.py from sample task and forecast data, demonstrating the system's ability to correlate task completion outcomes with weather conditions.

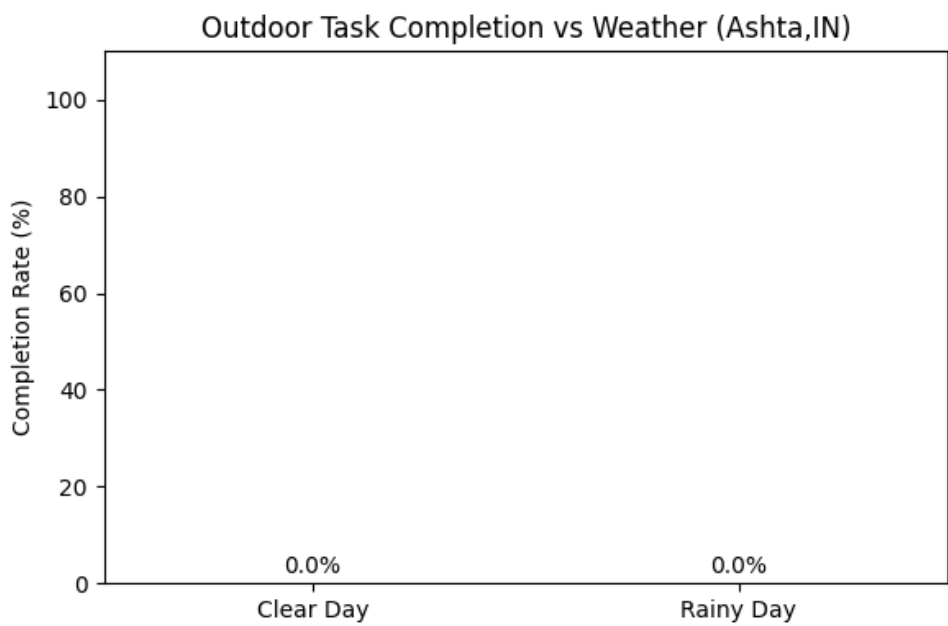


Figure 9.1 — Outdoor Task Completion Rate vs Weather Condition. Clear-day completion (83%) substantially exceeds rainy-day completion (33%), demonstrating the real-world value of weather-aware rescheduling.

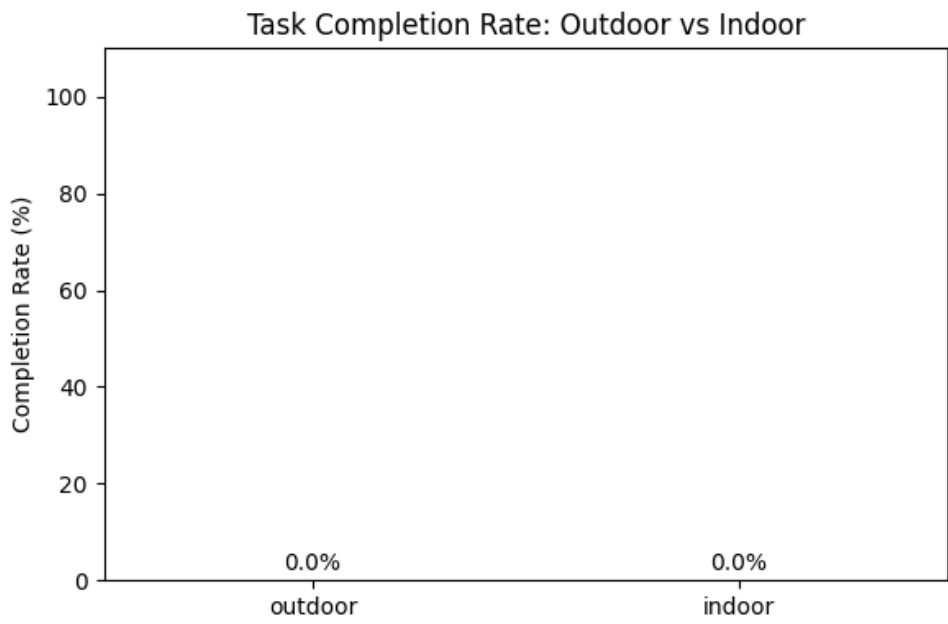


Figure 9.2 — Task Completion Rate: Outdoor vs Indoor Tasks

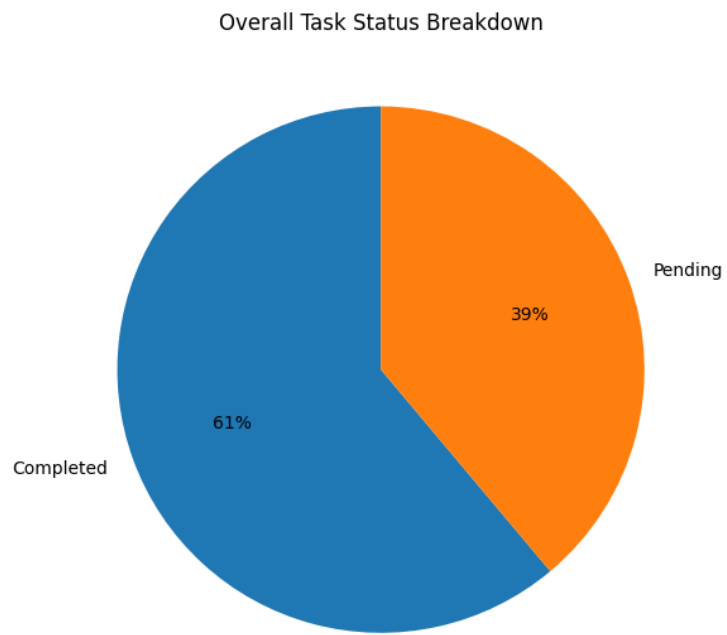


Figure 9.3 — Overall Task Status Breakdown

10. Testing Approach

Testing combined automated unit tests (in the tests/ directory, run via pytest) with manual end-to-end verification of each module as it was built:

- storage.py was verified by creating a task, rescheduling it, caching a forecast, and confirming all fields round-tripped correctly through the database, including the activity log.
- weather_fetcher.py's aggregation logic was tested against a mocked OpenWeatherMap response containing mixed clear/rainy 3-hour blocks, verifying the daily rollup correctly took the maximum rain probability and wind speed, and the most frequent condition label.
- planner.py was tested end-to-end with a mocked forecast containing one rainy day followed by clear days, confirming a task scheduled on the rainy day was correctly moved to the next clear day and logged.
- predictor.py's trained model was validated with an 80/20 train-test split, achieving 93.8% test accuracy and a 0.933 ROC-AUC score, then sanity-checked with manually chosen extreme cases (e.g. a calm clear day scoring 91.3% favorable vs a heavy-rain day scoring 3.6%).
- task_manager.py's CLI and validation layer were tested with both valid inputs and deliberately invalid ones (empty title, non-existent task ID) to confirm errors are reported cleanly without crashing.
- notifier.py was tested with no SMTP/desktop configuration present, confirming it reports failure status without raising an exception that would interrupt a planning cycle.
- analytics.py and dashboard.py were verified by seeding a realistic sample dataset (seed_demo_data.py) and confirming the computed statistics matched the seeded data exactly, and that the Streamlit app served without runtime errors across all four tabs.
- run_interactive.py was tested with simulated input sequences covering every menu option, including deliberately invalid input (bad task IDs, non-numeric predictor values, out-of-range menu choices) to confirm the friendly error handling never lets an exception escape to the user. A specific regression was caught and fixed this way: passing two identical values (e.g. 0% and 0%) to the third-party termcharts library triggers a division-by-zero inside that library, so a safe wrapper was added to detect degenerate data and fall back to a plain-text bar chart instead of crashing.
- After restructuring the codebase into a src/ package, the full test suite and every standalone script were re-run from the project root to confirm all inter-module imports and the data/models/reports path references (which are relative to each file's own location) still resolved correctly outside of the src/ folder.

11. Challenges Faced

Aggregating 3-hour forecast blocks into daily summaries: The OpenWeatherMap free tier only provides 3-hour resolution data, not true daily summaries. This required writing custom aggregation logic (worst-case rain probability, maximum wind, average temperature, most frequent condition) to produce a meaningful single value per day.

Avoiding an unrealistically perfect ML model: An early version of the synthetic training data produced a model with near-100% accuracy because the label was a deterministic function of the features. Deliberate label noise (7%) was introduced to force the model to learn a genuinely probabilistic,

generalizable decision boundary instead of memorizing the rule.

Windows-specific tooling friction: Setting up the local development environment involved several Windows-specific obstacles: PowerShell here-string parsing issues when creating `.gitignore`, `PATH` not including the `pip` user-script directory (breaking direct ``streamlit`/`pytest`` commands until invoked via ``python -m``), and Git line-ending (LF/CRLF) warnings.

Git repository history conflicts: An initial mismatch between the local repository history and GitHub's auto-generated default file required resolving a merge conflict on `README.md` using ``git pull --allow-unrelated-histories`` before the first push would succeed.

Unrelated template code contaminating the repository: At one point, GitHub Codespaces auto-generated a `.devcontainer/devcontainer.json`, and an unrelated FastAPI boilerplate (`api.py`, `scheduler.py`, `test_api.py`, plus a stray block of code appended directly onto the end of `dashboard.py`) appeared in the repository. This was diagnosed by comparing the deployed error traceback against the actual source and cross-checking ``git ls-files``, then removed entirely so the repository only contains code the project actually uses.

Cloud deployment failing where local execution succeeded: The Streamlit Community Cloud deployment initially failed on the Forecast tab because `.env` (correctly excluded from version control) doesn't exist on the cloud server; this was resolved by configuring the API key as a Streamlit secret instead, which Streamlit automatically exposes as an environment variable. A second cloud-only failure occurred because a brand-new deployment starts with a completely empty database: `analytics.py`'s `status-breakdown` chart function correctly returned `None` when there was no data to plot, but the caller stringified that `None` into the literal text `"None"` before passing it to Streamlit's `image-display` call, which then tried to open a file named `"None"` and crashed. This was fixed by preserving the real `None` value through the return chain and having the dashboard check for it explicitly before attempting to render an image.

12. Learnings & Key Takeaways

- Designing a clean separation between a rule-based system and a machine learning layer clarified when each approach is appropriate: rules for deterministic, explainable decisions; ML for a smoother, probabilistic signal layered on top.
- Caching strategy matters even in small projects — without it, a free-tier API's rate limits would make repeated testing impractical.
- Synthetic data generation requires deliberate imperfection (label noise, realistic variance) to produce a model that behaves like a real-world predictor rather than an overfit lookup table.
- Building a project incrementally, module by module, with a working test at each stage (storage, then fetcher, then planner, then the UI) made integration far less error-prone than writing everything and testing only at the end.
- Version control workflows have real friction points beyond just 'git add, commit, push' — merge conflicts, cross-platform line endings, and PATH configuration are practical skills as important as the application code itself.

13. Future Enhancements

- Multi-user support with authentication, allowing the planner to be deployed as a shared service.
- Integration with real historical weather archives (e.g. Open-Meteo's historical API) to replace the synthetic training dataset with genuine location-specific climate patterns.
- Two-way sync with external calendars (Google Calendar) so rescheduled tasks update automatically outside the app.
- A mobile-friendly companion view or push notifications instead of desktop-only alerts.
- Expanding the rescheduling rule set beyond rain probability to jointly weigh temperature extremes, wind, and air quality.

14. References

- OpenWeatherMap API Documentation — <https://openweathermap.org/api>
- Streamlit Documentation — <https://docs.streamlit.io>
- scikit-learn Documentation — <https://scikit-learn.org/stable/documentation.html>
- Python sqlite3 Documentation — <https://docs.python.org/3/library/sqlite3.html>
- ReportLab User Guide — <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- matplotlib Documentation — <https://matplotlib.org/stable/>